Engenharia Reversa — Oficina AutoCerta

Documentação técnica do sistema web fornecido

Escopo: requisitos, regras de negócio, DER, API, front-end, segurança, evidências de inspeção, fluxo funcional e limites de teste.

Resumo do sistema

Aplicação de oficina mecânica com front-end HTML/CSS/JavaScript, API Node.js/Express e banco SQLite. O sistema mantém usuários, clientes, veículos e agendamentos. O login usa sessão; senhas são comparadas com bcrypt e o CPF é cifrado com AES-256-GCM.

1. Requisitos funcionais

• RF01 autenticar usuário; RF02 encerrar sessão; RF03 consultar usuário logado.

• RF04 listar e buscar clientes por nome, CPF, telefone ou e-mail; RF05 cadastrar clientes; RF06 editar clientes; RF07 impedir CPF duplicado.

• RF08 listar veículos; RF09 cadastrar veículo associado a cliente existente; RF10 validar placa e impedir duplicidade.

• RF11 listar agendamentos com dados relacionados; RF12 criar agendamento; RF13 alterar data de agendamento.

• RF14 exibir mensagens e redirecionar para login quando a sessão expira.

Regras de negócio

• Rotas de dados exigem sessão; sessão configurada para 30 minutos e cookie httpOnly.

• Cliente exige nome e CPF com 11 dígitos; e-mail, quando informado, deve ter formato básico válido.

• CPF é único e precisa ser descriptografado para comparação devido ao IV aleatório.

• Placa é normalizada para maiúsculas e validada por expressão regular; placa é única no banco.

• Veículo exige cliente existente; agendamento exige veículo existente, data real no formato AAAA-MM-DD e serviço preenchido.

• Não há exclusão de registros, edição de veículo nem prevenção de conflitos de agendamento.

2. DER do banco

Tabelas: usuarios(id PK, nome, login UNIQUE, senha); clientes(id PK, nome, cpf, telefone, email); veiculos(id PK, placa UNIQUE, modelo, tipo, marca, cliente_id FK); agendamentos(id PK, veiculo_id FK, data_agendamento, servico). Um cliente possui vários veículos (1:N); um veículo possui vários agendamentos (1:N).

Veja também o arquivo docs/der.png no repositório.

3. Mapa da API

Método/rota	Entrada	Resposta e validação
POST /api/login	login, senha	200 ok; 400 campos ausentes; 401 credenciais inválidas
POST /api/logout	cookie opcional	Destrói sessão e retorna ok
GET /api/me	cookie sessão	200 id/nome; 401 sem sessão
GET /api/clientes?busca=	busca opcional	Lista e filtra nome/e-mail/telefone/CPF
POST /api/clientes	nome, cpf, telefone?, email?	201; 400 por validação/duplicidade
PUT /api/clientes/:id	id + campos cliente	200; 404 inexistente; 400 dados inválidos
GET /api/veiculos	cookie sessão	Lista veículos com nome do cliente
POST /api/veiculos	placa, modelo, tipo, marca?, cliente_id	201; 400 placa/campos/cliente inválidos
GET /api/agendamentos	cookie sessão	Lista agendamentos com JOINs e ordenação por data
POST /api/agendamentos	veiculo_id, data_agendamento, servico	201; 400 veículo/data/serviço inválido
PUT /api/agendamentos/:id	id, data_agendamento	200; 404 id inexistente; 400 data inválida

4. Mapa do front-end

Página	Scripts	Principais rotas
login.html	comum.js, login.js	POST /api/login
principal.html	comum.js + iniciarPagina()	GET /api/me; POST /api/logout

Página	Scripts	Principais rotas
clientes.html	comum.js, clientes.js	GET /api/clientes; POST; PUT /api/clientes/:id
veiculos.html	comum.js, veiculos.js	GET /api/veiculos; GET /api/clientes; POST /api/veiculos
agendamentos.html	comum.js, agendamentos.js	GET /api/agendamentos; GET /api/veiculos; POST; PUT /api/agendamentos/:id

5. Análise de segurança

Pontos positivos: bcrypt para senhas; AES-256-GCM para CPF; cookie httpOnly; middleware de sessão nas rotas de dados; consultas parametrizadas; escape de texto antes de inserir valores em innerHTML.

• Risco 1: segredo da sessão e derivação da chave de CPF fixos no código. Melhoria: usar variáveis de ambiente ou Secret Manager e planejar rotação.

• Risco 2: cookie não configura sameSite/secure e não há proteção CSRF explícita. Melhoria: configurar política de cookie, HTTPS em produção e proteção CSRF.

• Risco 3: sem limitação de tentativas de login. Melhoria: rate limiting, atraso progressivo e monitoramento.

• Outros: CPF valida apenas 11 dígitos, não dígitos verificadores; CPF é devolvido em claro para a interface; armazenamento padrão de sessão não é adequado para produção.

6. Casos de teste — resultado observado

Os casos abaixo são verificações reais de esquema/dados e inspeções estáticas. Não equivalem a teste ponta a ponta HTTP: o servidor não iniciou neste ambiente porque o pacote express não estava disponível, e npm install não concluiu.

ID	Caso	Método	Resultado
CT01	Existência das quatro tabelas	Consulta SQLite sqlite_master	PASSOU: usuarios, sqlite_sequence, clientes, veiculos, agendamentos
CT02	Quantidade inicial em usuarios	SELECT COUNT(*)	PASSOU: 3 registros (esperado 3)
CT03	Quantidade inicial em clientes	SELECT COUNT(*)	PASSOU: 3 registros (esperado 3)
CT04	Quantidade inicial em veiculos	SELECT COUNT(*)	PASSOU: 4 registros (esperado 4)
CT05	Quantidade inicial em agendamentos	SELECT COUNT(*)	PASSOU: 4 registros (esperado 4)
CT06	FK veículos -> clientes	PRAGMA foreign_key_list(veiculos)	PASSOU: [(0, 0, 'clientes', 'cliente_id', 'id', 'NO ACTION', 'NO ACTION', 'NONE')]
CT07	FK agendamentos -> veículos	PRAGMA foreign_key_list(agendamentos)	PASSOU: [(0, 0, 'veiculos', 'veiculo_id', 'id', 'NO ACTION', 'NO ACTION', 'NONE')]
CT08	Unicidade da placa declarada no esquema	Inspeção de oficina_db.sql	PASSOU: UNIQUE declarado na coluna placa
CT09	Hash bcrypt usado no login	Inspeção de server.js	PASSOU: Comparação com bcrypt.compareSync
CT10	Sessão expira em 30 minutos	Inspeção de server.js	PASSOU: maxAge configurado para 30 minutos
CT11	Rotas de dados usam middleware de login	Inspeção de server.js	PASSOU: Rotas consultadas incluem exigirLogin
CT12	CPF usa AES-256-GCM	Inspeção de seguranca.js	PASSOU: Algoritmo AES-256-GCM declarado
CT13	Escape de HTML no front-end	Inspeção de comum.js	PASSOU: Função esc() codifica caracteres HTML
CT14	Placa é normalizada para maiúsculas	Inspeção de server.js	PASSOU: Normalização e regex presentes
CT15	Validação rejeita data impossível	Inspeção de server.js	PASSOU: Validação compara data ISO de retorno
CT16	Agendamentos incluem JOIN com veículos e clientes	Inspeção de server.js	PASSOU: Consulta possui os dois JOINs
CT17	Busca de clientes contempla nome/e-mail/telefone/CPF	Inspeção de server.js	PASSOU: Quatro campos contemplados no filtro

Limitação registrada em docs/evidencias/limite_execucao.md. Para completar, executar npm install/npm start no computador local e testar as rotas com navegador, cURL ou Postman.

7. Fluxo — cadastro de veículo

Usuário abre Veículos → a tela solicita GET /api/veiculos → API verifica sessão → usuário clica Novo veículo → front-end carrega clientes → usuário envia POST /api/veiculos → API valida placa, campos obrigatórios e cliente existente → em erro, retorna 400; em sucesso, grava no SQLite e retorna 201 → front-end recarrega a lista.

8. Código comentado

A pasta sistema_comentado/ contém cópia do sistema com comentários sobre intenção das rotas, validações, criptografia, consultas relacionadas, escape de HTML e fluxo de sessão. Os comentários foram adicionados sem alterar deliberadamente as regras de negócio.

Fontes internas utilizadas: server.js, seguranca.js, db/conexao.js, db/oficina_db.sql, public/*.html e public/js/*.js fornecidos no ZIP.